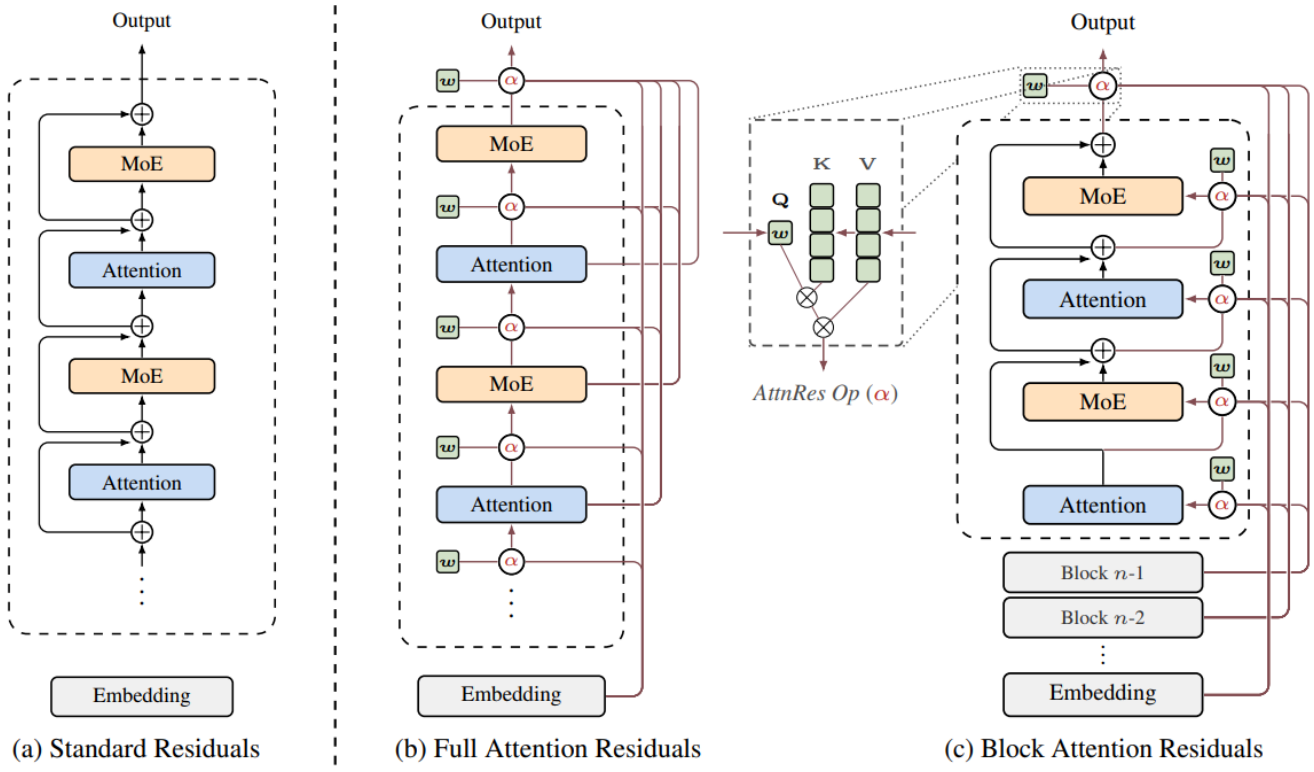
ATTENTION RESIDUALS

> #Architecture



- What problems does this architecture solve?
  - The original residual connection is a ground-breaking design in deep learning as it helps to avoid the vanishing gradient problem.
  - However, traditional residual connections are simply adding the previous layers' outputs together, which will cause the signal in the stream to increase with a rate of $O(L)$. The previous layer's output will have a lower and lower weight in the hidden state.
- What does this architecture do?
  - Instead of simple additions, the paper proposes an adaptive way of calculating a weight for each output.
  - The intuition is like what we do in the attention mechanism: for each layer, we store a **pseudo-query** vector $w_l \in \mathbb{R}^d$. Using the hidden state of that layer serves as the **key** and the **value**.
    - Before adding the previous outputs to the hidden state, we calculate the attention as the weight of that layer's output. Formally:

      $h_l = \alpha_{0\to l} \cdot h_1 + \sum_{i=1}^{l-1} \alpha_{i\to l} \cdot f_i(h_i)$,

      where $\alpha_{i\to l} = \frac{\exp(w_l^\top \mathrm{RMSNorm}(v_i))}{\sum_{j=0}^{l-1} \exp(w_l^\top \mathrm{RMSNorm}(v_j))}$,

      where $k_i = v_i = \begin{cases} h_1 & i = 0 \\ f_i(h_i) & 1 \le i \le l-1 \end{cases}$
  - Further optimization
    - If L, the total # of layers, is large, then we need to save $O(L \cdot d)$ pseudo-query vectors,

and the time cost will be $O(L^2 \cdot d)$. In Pipeline Parallelism, communicating L vectors can cause overhead.

- To avoid this, we divide the L layers into N blocks, where $N << L$. Treat each block as a layer, with the outputs of actual layers in a block simply added:

  $b_n = \sum_{j \in \mathcal{B}_n} f_j(h_j)$

- Each block will only attend to the previous blocks. For layers in the current block, it will also attend to the partially calculated output of the current block: b_n^{i-1}.
- Now, the communication cost/space cost will be $O(N \cdot d)$, and the time complexity is $O(N^2 \cdot d)$.

- Experiment Results
  - Scaling Laws: In tests across five different parameter scales, both Full Attention Residual and Block Attention Residual consistently outperformed baseline models across all computational power ranges. Notably, the performance of Block Attention Residual is equivalent to the baseline model consuming 1.25 times more training computational power.
  - Comprehensive improvements in downstream tasks: In tests using a 48B-parameter (3B-activation) large model pre-trained on 1.4T tokens, Block Attention Residual matched or outperformed the baseline model across all benchmarks, including language, reasoning, mathematics, and code generation. The improvements are particularly significant in tasks requiring combinatorial abilities, such as “multi-step reasoning” (e.g., GPQA-Diamond, Math) and “code generation.”
  - Healthier training dynamics: Experimental data demonstrate that this mechanism successfully resolves the “infinite feature amplitude inflation” issue caused by traditional PreNorm, keeping the output amplitude of each layer within a bounded range. Additionally, through dynamic attention competition, it significantly mitigates the problem of excessive gradient steepness in early layers found in traditional residual networks, resulting in a more uniform gradient distribution across the network depth.
  - Shift in architectural preferences: Ablation experiments not only demonstrate the critical importance of dynamic weights and RMSNorm normalization, but architectural search also reveals that, under equivalent computational and parameter budgets, models equipped with AttnRes benefit more from “deeper, narrower” network architectures than traditional Transformers.

- Work Cited
  - Paper
  - Chat History